



Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias
Ingeniería en Ciencia de Datos y Matemáticas

Documentación de Desarrollo — Gestor de Identidades v2.0

Casa Monarca — Ayuda Humanitaria al Migrante A.B.P

Profesores:

Raúl Gómez Muñoz Alberto Francisco Martínez Herrera
Pedro Leonel Olaya Trejos Daniel Otero Fadul

Equipo 1 — Grupo 602

Mariel Álvarez Salas A01198828
Brisma Alvarez Valdez A00839238
Emiliano Ruiz López A01659693
Ana Sofia Nagao Alvarez A01285034
Karen Aylen Estrada Ceferino A00838403

Monterrey, Nuevo León. 14 de junio de 2026

Índice general

1 Descripción General	5
1.1 Propósito del sistema	5
1.2 Alcance	5
1.3 Casos de uso principales	5
2 Arquitectura del Sistema	7
2.1 Tipo de arquitectura	7
2.2 Componentes principales	7
2.3 Diagrama general	8
3 Stack Tecnológico	9
4 Estructura del Proyecto	10
4.1 Organización de carpetas	10
4.2 Descripción de módulos	10
4.2.1 iam	10
4.2.2 registros	11
4.2.3 crypto_core	11
5 Configuración del Entorno	12
5.1 Requisitos de versión	12
5.2 Variables de entorno	12
5.3 Archivos de configuración	12
6 Instalación y Ejecución	14
6.1 Pasos para levantar el sistema	14
6.2 Comandos principales	14
7 Base de Datos	15
7.1 Modelo de datos	15
7.2 Tablas principales	15
7.3 Relaciones clave	16
8 API / Interfaces (Rutas URL)	17
8.1 Módulo IAM	17
8.2 Módulo Registros	17
8.3 Ejemplo de uso: crear registro vía vistas	18
9 Flujos y Lógica de Negocio	19
9.1 Flujo de autenticación (login)	19
9.2 Flujo de onboarding de nuevo colaborador	20
9.3 Flujo del workflow de aprobación	21
9.4 Reglas importantes	21
10 Ejemplos de Uso	22
10.1 Crear un colaborador desde código	22

10.2 Emitir un certificado X.509	22
10.3 Firmar y verificar un expediente	22
10.4 Consultar el estado de la cadena de auditoría	23
11 Manejo de Errores	24
11.1 Estrategia de errores	24
11.2 Mensajes comunes	24
11.3 Registro (logs)	24
12 Pruebas	25
12.1 Tipos de tests	25
12.2 Cómo ejecutar las pruebas	25
13 Despliegue	26
13.1 Entornos	26
13.2 Cómo desplegar en producción (HostGator)	26
13.3 Consideraciones de producción	26
14 Seguridad	27
14.1 Autenticación	27
14.2 Manejo de datos sensibles	27
14.3 Buenas prácticas implementadas	27
15 Guía de Contribución	28
15.1 Convenciones de código	28
15.2 Flujo de trabajo con git	28
15.3 Cómo agregar una nueva funcionalidad	28
16 Problemas Conocidos	29
16.1 Bugs actuales	29
16.2 Limitaciones	29
17 FAQ Técnica	30
Referencias	31

Índice de cuadros

1	Casos de uso principales por módulo (elaboración propia).	6
2	Componentes del sistema y su responsabilidad (elaboración propia).	7
3	Stack tecnológico completo (elaboración propia).	9
4	Requisitos del entorno de desarrollo y producción (elaboración propia).	12
5	Variables de entorno del sistema (elaboración propia).	12
6	Comandos de gestión más usados (elaboración propia).	14
7	Tablas principales del sistema (elaboración propia).	15
8	Rutas del módulo IAM (elaboración propia).	17
9	Rutas del módulo Registros (elaboración propia).	18
10	Reglas de escalación por tipo de acción (elaboración propia).	21
11	Mensajes de error frecuentes y su causa (elaboración propia).	24
12	Entornos de despliegue (elaboración propia).	26

Índice de figuras

1	Diagrama de arquitectura MVT del sistema, ilustrando la separación entre el módulo IAM (gestión de colaboradores, onboarding, certificados X.509 y auditoría), el módulo Registros (expedientes cifrados con AES-256-GCM, flujo de aprobación y solicitudes ARCO) y la capa criptográfica transversal <code>crypto_core</code> . La comunicación cliente–servidor sigue el patrón HTTP síncrono de Django; la base de datos es MySQL en producción y SQLite en desarrollo. Generado con Claude (Anthropic, 2026).	8
2	Flujo de autenticación del sistema. Las validaciones se ejecutan en secuencia: (1) bloqueo por exceso de intentos fallidos (Error 429), (2) autenticación de credenciales con <code>authenticate()</code> de Django, (3) verificación del estado activo de la cuenta, (4) redirección al onboarding si está pendiente, (5) comprobación del certificado X.509 únicamente para Niveles 1–2 (fingerprint, estado y vigencia), y (6) validación del código TOTP cuando MFA está habilitado. Cada rama de error termina el proceso; la rama “No” en cada decisión continúa el flujo principal. Elaboración propia.	20

Descripción General

1.1 Propósito del sistema

El sistema de Gestión de Identidades y Registro de Migrantes de Casa Monarca es una aplicación web interna desarrollada en Django cuyo propósito es doble:

1. **Módulo IAM (Identity & Access Management):** controlar el ciclo de vida de los colaboradores de Casa Monarca. Emite y revoca certificados digitales X.509 para los niveles de mayor privilegio, lleva un registro de auditoría de todas las acciones y detecta intentos de acceso no autorizados.
2. **Módulo de Registro de Migrantes:** gestionar expedientes de personas beneficiarias con cifrado AES-256 de los campos personales sensibles, firma ECDSA por expediente, un flujo de aprobación escalonado por nivel de acceso y herramientas para ejercer los derechos ARCO conforme a la LFPDPPP.

1.2 Alcance

Lo que el sistema hace:

- Alta, baja, suspensión y revocación de colaboradores (Niveles 1–4).
- Emisión y revocación de certificados X.509 para Niveles 1 y 2.
- Onboarding con validación manual por parte del administrador.
- Registro de expedientes de migrantes con cifrado de datos personales.
- Firma ECDSA por expediente y cadena de auditoría criptográfica.
- Flujo de aprobación escalonado: Nivel 4 → 3 → 2 → 1.
- Gestión completa de los cuatro derechos ARCO con plazos legales calculados.
- Auditoría persistente e inalterable de todas las acciones del sistema.
- Recuperación de contraseña mediante pregunta de seguridad.
- Autenticación TOTP (MFA) opcional para Niveles 1 y 2.

Limitaciones y fuera de alcance:

- El sistema es una aplicación web interna; no hay API REST pública ni mobile app.
- No implementa notificaciones por correo electrónico en esta versión.
- No hay mecanismo de respaldo automático de base de datos (pendiente como mejora futura).
- La CA es autofirmada (self-signed); no está integrada con una autoridad certificadora externa.
- El servidor HostGator no soporta Java ni Docker, por lo que soluciones como Keycloak o step-ca quedan fuera del entorno de producción.

1.3 Casos de uso principales

Cuadro 1: Casos de uso principales por módulo (elaboración propia).

Módulo	Actor	Caso de uso
IAM	Admin (N1)	Alta/baja/revocación de colaborador; emisión/revocación de certificado
IAM	Colaborador	Onboarding; descarga de credenciales; recuperación de contraseña
IAM	Admin (N1)	Auditoría del sistema; gestión de certificados activos
Registros	N1/N2	Crear/editar/eliminar registro de migrante directamente
Registros	N3/N4	Solicitar creación o edición vía flujo de aprobación
Registros	N1/N2	Aprobar, rechazar o ejecutar solicitudes del workflow
Registros	N3+	Crear solicitud ARCO para un migrante
Registros	N2/N1	Revisar, aprobar y ejecutar una solicitud ARCO

Arquitectura del Sistema

2.1 Tipo de arquitectura

El sistema sigue la arquitectura MVT (*Model–View–Template*) de Django, una variante del patrón MVC donde:

- **Model:** define la estructura de datos y las reglas de negocio en `models.py`.
- **View:** contiene la lógica de cada petición HTTP en `views.py`; valida permisos, llama a los modelos y retorna respuestas.
- **Template:** las plantillas HTML en `templates/` renderizan la interfaz a partir del contexto enviado por las vistas.

La comunicación cliente–servidor es síncrona (peticiones HTTP convencionales); no se emplea AJAX salvo para marcar notificaciones como leídas. La capa criptográfica está aislada en el paquete `crypto_core/`.

2.2 Componentes principales

Cuadro 2: Componentes del sistema y su responsabilidad (elaboración propia).

Componente	Responsabilidad
<code>casamonarca/</code>	Proyecto Django raíz: configuración, URL dispatcher principal
<code>iam/</code>	App IAM: modelos, vistas, URLs, lógica de onboarding y certificados
<code>registros/</code>	App registros: modelos CRUD, workflow, ARCO, tickets, cadena hash
<code>crypto_core/</code>	Librería interna: cifrado AES-256, firmas ECDSA, CA, hashing, campos personalizados
<code>templates/</code>	Plantillas HTML organizadas por app
<code>static/</code>	CSS, JS e imágenes estáticas
<code>media/</code>	Archivos subidos por usuarios (IDs oficiales, documentos ARCO)
<code>tests/</code>	Suite de pruebas de integración criptográfica

2.3 Diagrama general

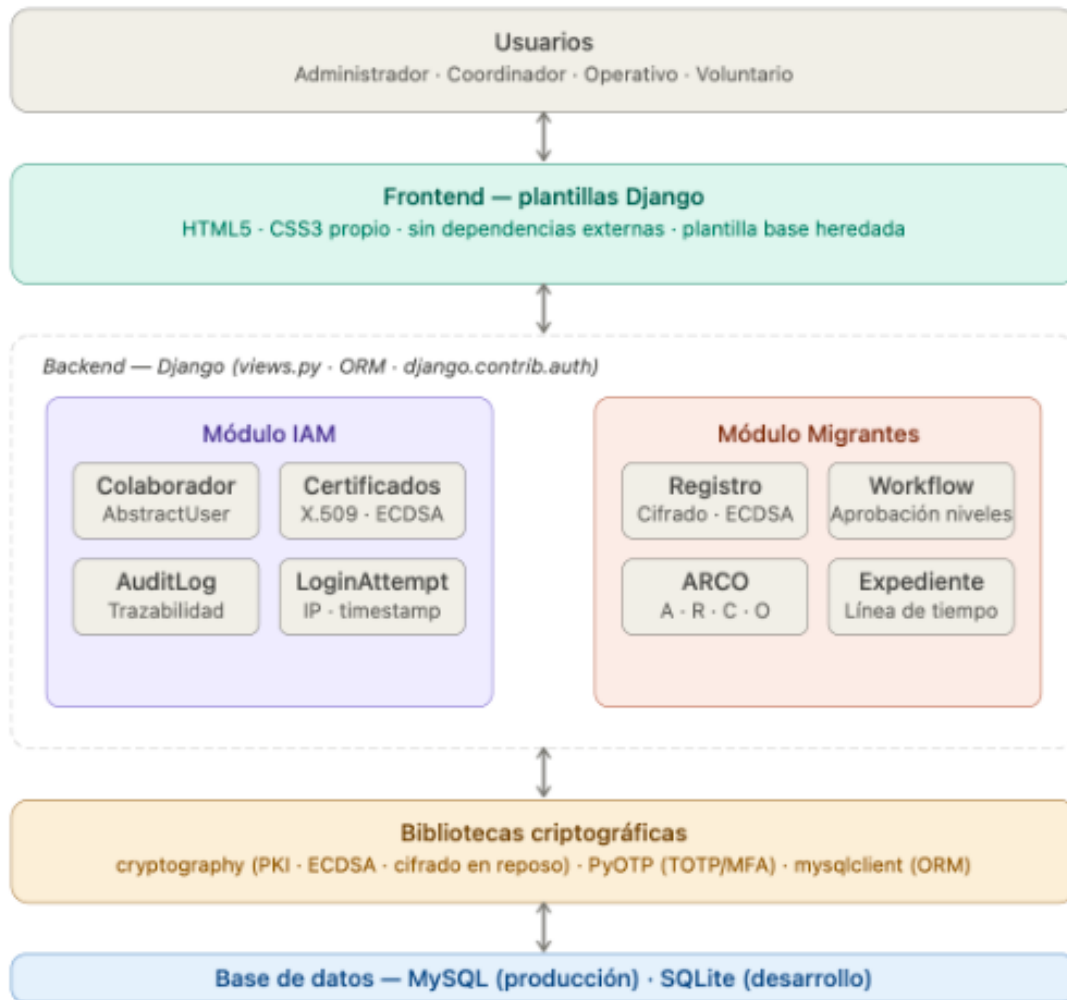


Figura 1: Diagrama de arquitectura MVT del sistema, ilustrando la separación entre el módulo **IAM** (gestión de colaboradores, onboarding, certificados X.509 y auditoría), el módulo **Registros** (expedientes cifrados con AES-256-GCM, flujo de aprobación y solicitudes ARCO) y la capa criptográfica transversal `crypto_core`. La comunicación cliente–servidor sigue el patrón HTTP síncrono de Django; la base de datos es MySQL en producción y SQLite en desarrollo. Generado con Claude (Anthropic, 2026).

Stack Tecnológico

Cuadro 3: Stack tecnológico completo (elaboración propia).

Categoría	Tecnología	Versión / Uso
Lenguaje	Python	3.10 o superior
Framework web	Django	≥ 5.0 , < 7.0
Base de datos (prod)	MySQL	8.0+ vía PyMySQL ≥ 1.0
Base de datos (dev)	SQLite	Incluida en Python; sin configuración
Criptografía	cryptography	≥ 41.0 (AES-256-GCM, ECDSA, X.509)
MFA	pyotp	≥ 2.0 (TOTP RFC 6238)
JWT	PyJWT	$\geq 2.8.0$ (tokens auxiliares)
Variables de entorno	python-dotenv	≥ 1.0
Pruebas	pytest + pytest-django	≥ 7.0 y ≥ 4.5
Teléfonos	phonenumbers	Última estable
Servidor producción	Gunicorn + nginx	Recomendado para HostGator

Estructura del Proyecto

4.1 Organización de carpetas

```
CriptoReto/
|-- casamonarca/ # Proyecto Django raiz
| |-- settings.py # Configuración central
| |-- urls.py # URL dispatcher principal
| '-- wsgi.py / asgi.py
|-- iam/ # App: gestión de identidades
| |-- models.py # Collaborator, UserCertificate, AuditLog, LoginAttempt
| |-- views.py # Vistas de login, onboarding, certificados
| |-- urls.py # 42 rutas del módulo IAM
| '-- management/commands/ # Comandos personalizados
|-- registros/ # App: registros de migrantes
| |-- models.py # 12 modelos (MigrantRegistration, WorkflowRequest, etc.)
| |-- views.py # Vistas CRUD, workflow, ARCO
| |-- urls.py # 50+ rutas del módulo registros
| '-- workflow.py # Lógica de escalación del flujo de aprobación
|-- crypto_core/ # Librería criptográfica interna
| |-- encryption.py # AES-256-GCM (EncryptionService)
| |-- signatures.py # ECDSA con SECP256K1
| |-- ca.py # Autoridad Certificadora interna
| |-- certificates.py # Emisión y verificación de certificados X.509
| |-- fields.py # EncryptedTextField, EncryptedDateField
| '-- hashing.py # SHA-256 y utilidades de hash
|-- templates/ # Plantillas HTML
| |-- iam/ # Pantallas del módulo IAM
| |-- registros/ # Pantallas del módulo registros
| '-- partials/ # Componentes reutilizables (base, navbar, etc.)
|-- static/ # CSS, JS, imágenes estáticas
|-- media/ # Archivos subidos (IDs oficiales, PDFs ARCO)
| |-- official_ids/
| |-- arco_docs/
| '-- arco_exports/
|-- tests/
| '-- test_crypto_integration.py
|-- scripts/ # Scripts auxiliares (seed, migraciones iniciales)
|-- seed.py # Poblar BD con datos de prueba
|-- requirements.txt
'-- manage.py
```

4.2 Descripción de módulos

4.2.1 iam

Gestiona el ciclo de vida de los colaboradores. Contiene los modelos `Area`, `Collaborator` (extiende `AbstractUser`), `UserCertificate`, `CertificateAuditLog`, `AuditLog` y `LoginAttempt`. La lógica de control de acceso está encapsulada en métodos de instancia del modelo `Collaborator` (`can_view`, `can_edit`, `can_issue_certificate`, etc.).

4.2.2 registros

Gestiona los expedientes de migrantes y los flujos de aprobación. Contiene 12 modelos: `MigrantRegistration`, `MigrantRegistrationSignature`, `ActionSignature`, `BatchSignSession`, `WorkflowRequest`, `ApprovalStep`, `Ticket`, `ArcoRequest`, `ArcoTicket`, `Notification`, `RegistrationEvent` y `SignedFlowLog`. El archivo `workflow.py` encapsula las reglas de escalación.

4.2.3 crypto_core

Librería criptográfica interna que ningún código externo debería reimplementar. Expone:

- `EncryptionService`: cifrado y descifrado AES-256-GCM.
- `ECDSASigner`: firma y verificación ECDSA sobre `secp256k1`.
- `CertificateAuthority`: generación y verificación de certificados X.509.
- `EncryptedTextField` / `EncryptedDateField`: campos Django con cifrado transparente en el ORM.
- `sha256_hex`: función auxiliar SHA-256.

Configuración del Entorno

5.1 Requisitos de versión

Cuadro 4: Requisitos del entorno de desarrollo y producción (elaboración propia).

Componente	Versión mínima	Notas
Python	3.10	Se recomienda 3.11 o 3.12
pip	23.0+	Incluido con Python
MySQL	8.0	Solo en producción; SQLite en desarrollo
Git	2.30+	Para clonar el repositorio
Sistema operativo	macOS 12 / Ubuntu 20.04 / Windows 10+WSL	HostGator usa cPanel con Python

5.2 Variables de entorno

El sistema lee variables de entorno mediante `python-dotenv`. En desarrollo se utiliza un archivo `.env` en la raíz del proyecto. En producción deben configurarse en el entorno del servidor (cPanel → Variables de entorno).

Cuadro 5: Variables de entorno del sistema (elaboración propia).

Variable	Obligatoria	Descripción
SECRET_KEY	Sí (prod)	Clave secreta de Django. Generar con <code>django-admin startproject 0 secrets.token_urlsafe(50)</code> .
DEBUG	No	True en desarrollo, False en producción.
ALLOWED_HOSTS	Sí (prod)	Lista de dominios permitidos, p.ej. <code>casamonarca.org</code> .
MYSQL_DATABASE	Sí (prod)	Nombre de la base de datos MySQL. Si no está definida, usa SQLite.
MYSQL_USER	Prod	Usuario MySQL (<i>default</i> : <code>casamonarca_user</code>).
MYSQL_PASSWORD	Prod	Contraseña MySQL.
MYSQL_HOST	Prod	Host MySQL (<i>default</i> : <code>127.0.0.1</code>).
MYSQL_PORT	No	Puerto MySQL (<i>default</i> : <code>3306</code>).
CERT_ENCRYPTION_KEY	Sí (prod)	Clave base64 de 32 bytes para cifrar datos de certificados. En <code>DEBUG=True</code> se deriva de <code>SECRET_KEY</code> .
CERT_ISSUER_NAME	No	Nombre del emisor X.509 (<i>default</i> : <code>Casa Monarca IAM</code>).
CERT_EXPIRATION_DAYS	No	Vigencia de certificados en días (<i>default</i> : <code>365</code>).
CERT_PRIVATE_KEY_PATH	Sí (prod)	Ruta a la clave privada PEM de la CA interna.
ECC_PRIVATE_KEY_PATH	Sí (prod)	Ruta a la clave ECC PEM utilizada para firmas del servidor.

5.3 Archivos de configuración

`casamonarca/settings.py` — configuración central de Django. No editar directamente en producción; usar variables de entorno.

`.env` (no subir al repositorio) — ejemplo mínimo para desarrollo:

```
SECRET_KEY=django-dev-only-key-change-in-production
DEBUG=True
# Dejar MYSQL_DATABASE sin definir para usar SQLite en desarrollo
```

```
# CERT_ENCRYPTION_KEY se deriva automaticamente en DEBUG=True
CERT_ISSUER_NAME=Casa Monarca IAM (Dev)
CERT_EXPIRATION_DAYS=365
ECC_PRIVATE_KEY_PATH=./ecc_signing_key.pem
```

requirements.txt — dependencias pinadas por versión mínima:

```
Django>=5.0,<7.0
PyMySQL>=1.0
pyotp>=2.0
cryptography>=41.0
PyJWT>=2.8.0
python-dotenv>=1.0
pytest>=7.0
pytest-django>=4.5
phonenumbers
```

Instalación y Ejecución

6.1 Pasos para levantar el sistema

Paso 1 — Clonar el repositorio:

```
git clone https://github.com/[organizacion]/Casa-Monarca-Criptografia.git
cd Casa-Monarca-Criptografia/CriptoReto
```

Paso 2 — Crear y activar el entorno virtual:

```
python -m venv venv
source venv/bin/activate # macOS / Linux
# venv\Scripts\activate # Windows
```

Paso 3 — Instalar dependencias:

```
pip install -r requirements.txt
```

Paso 4 — Configurar variables de entorno:

```
cp .env.example .env # si existe, o crear manualmente
# Editar .env con los valores reales
```

Paso 5 — Aplicar migraciones:

```
python manage.py migrate
```

Paso 6 — Crear administrador inicial:

```
python manage.py createsuperuser
```

Paso 7 — (Opcional) Cargar datos semilla:

```
python seed.py
```

Paso 8 — Iniciar el servidor de desarrollo:

```
python manage.py runserver
```

El sistema queda disponible en <http://127.0.0.1:8000>.

6.2 Comandos principales

Cuadro 6: Comandos de gestión más usados (elaboración propia).

Comando	Descripción
python manage.py migrate	Aplica migraciones pendientes
python manage.py makemigrations	Genera nuevas migraciones a partir de cambios en modelos
python manage.py createsuperuser	Crea el primer administrador (Nivel 1)
python manage.py collectstatic	Reúne estáticos para producción
python manage.py shell	REPL de Django con todos los modelos cargados
pytest	Ejecuta la suite completa de pruebas
pytest tests/ -v	Pruebas con salida detallada

Base de Datos

7.1 Modelo de datos

El sistema tiene 18 tablas distribuidas en dos aplicaciones Django. Los nombres en la base de datos siguen la convención <app>_<model> (p.ej. iam_collaborator, registros_migrantregistration).

7.2 Tablas principales

Cuadro 7: Tablas principales del sistema (elaboración propia).

Tabla	App	Descripción
iam_collaborator	iam	Extiende auth_user. Contiene nivel de acceso, estado de onboarding, campos de revocación, TOTP y pregunta de seguridad.
iam_usercertificate	iam	Certificado X.509 de un colaborador. Relación 1:1 con Collaborator. Almacena datos cifrados, huella digital, estado y fechas del ciclo de vida.
iam_certificateauditlog	iam	Registro inmutable de cada cambio de estado de un certificado. No permite UPDATE ni DELETE.
iam_auditlog	iam	Bitácora general del sistema: actor, objetivo, acción, timestamp.
iam_loginattempt	iam	Registro de cada intento de inicio de sesión con IP y resultado.
registros_migrantregistration	registros	Expediente del migrante. Campos PII (nombre, apellidos, fecha nacimiento, teléfono, país) cifrados con AES-256-GCM.
registros_migrantregistrationsignature	registros	Firma ECDSA del expediente. Relación 1:1. Guarda (r, s) , clave pública, curva, firmante.
registros_actionsignature	registros	Firma de cualquier acción del sistema. Encajonada por prev_chain_hash (SHA-256).
registros_workflowrequest	registros	Solicitud de creación/edición/eliminación que requiere aprobación jerárquica.
registros_approvalstep	registros	Decisión individual dentro de un WorkflowRequest. Append-only con cadena de hash.
registros_arcorequest	registros	Solicitud ARCO formal con plazo legal y documentos adjuntos.
registros_arcoticket	registros	Flujo de autorización ARCO-específico (coordinador → administrador).
registros_ticket	registros	Ticket genérico creado automáticamente al crear un registro o workflow.
registros_registrationevent	registros	Bitácora por expediente: consultas, ediciones, consentimientos, ARCO.
registros_signedflowlog	registros	Log inmutable de acciones irreversibles firmadas con X.509. No permite UPDATE ni DELETE.
registros_notification	registros	Notificación interna enviada al responsable de un workflow o ARCO.

7.3 Relaciones clave

- `Collaborator (1) ↔ (0..1) UserCertificate` — un colaborador puede tener un certificado activo o ninguno.
- `UserCertificate (1) → (N) CertificateAuditLog` — cada cambio de estado genera un registro append-only.
- `MigrantRegistration (1) ↔ (1) MigrantRegistrationSignature` — todo expediente tiene exactamente una firma ECDSA.
- `MigrantRegistration (1) → (N) WorkflowRequest` — puede tener múltiples solicitudes de cambio a lo largo del tiempo.
- `WorkflowRequest (1) → (N) ApprovalStep` — cada decisión de cada aprobador queda registrada.
- `ArcoRequest (1) ↔ (1) ArcoTicket` — cada solicitud ARCO tiene su ticket de seguimiento.
- `ActionSignature` — cadena encadenada mediante `prev_chain_hash`; el primer registro tiene `prev_chain_hash=.`

API / Interfaces (Rutas URL)

8.1 Módulo IAM

Prefijo base: /iam/ (definido en casamonarca/urls.py).

Cuadro 8: Rutas del módulo IAM (elaboración propia).

Ruta	Métodos	Descripción
/iam/login/	GET, POST	Formulario de inicio de sesión con .cert/.key para N1–N2
/iam/logout/	POST	Cierre de sesión
/iam/	GET	Dashboard con estadísticas del IAM
/iam/users/add/	GET, POST	Crear nuevo colaborador (solo Admin)
/iam/users/<pk>/	GET	Detalle de colaborador
/iam/users/<pk>/edit/	GET, POST	Editar colaborador
/iam/users/<pk>/toggle/	POST	Activar/desactivar cuenta
/iam/users/<pk>/revoke/	POST	Revocar acceso
/iam/users/<pk>/delete/	POST	Eliminación lógica
/iam/users/<pk>/certificate/issue/	GET, POST	Emitir certificado X.509
/iam/users/<pk>/certificate/validate/	GET	Verificar validez del certificado
/iam/users/<pk>/certificate/revoke/	POST	Revocar certificado
/iam/onboarding/	GET, POST	Formulario de onboarding del propio usuario
/iam/onboarding/download/	GET	Descargar .cert
/iam/onboarding/download/key/	GET	Descargar .key
/iam/onboarding/pending/	GET	Lista de onboardings pendientes (Admin)
/iam/users/<pk>/onboarding/approve/	POST	Aprobar onboarding
/iam/audit/	GET	Visor de auditoría con filtros
/iam/certificados/	GET	Gestión de certificados (Admin)
/iam/certificados/<id>/suspender/	POST	Suspender certificado
/iam/certificados/<id>/reactivar/	POST	Reactivar certificado suspendido
/iam/certificados/<id>/baja/	POST	Dar de baja un certificado
/iam/recovery/	GET, POST	Solicitar recuperación de contraseña
/iam/password/change/	GET, POST	Cambio de contraseña autenticado
/iam/api/user-role/	GET	JSON con nivel y tipo del usuario en sesión

8.2 Módulo Registros

Prefijo base: /registros/.

Cuadro 9: Rutas del módulo Registros (elaboración propia).

Ruta	Métodos	Descripción
/registros/	GET	Listado de expedientes activos
/registros/nuevo/	GET, POST	Formulario de nuevo registro
/registros/nuevo/revisar/	GET, POST	Revisión y confirmación con firma
/registros/<pk>/	GET	Detalle del expediente
/registros/<pk>/editar/	GET, POST	Editar registro (N1/N2 directo)
/registros/<pk>/eliminar/	POST	Eliminación lógica (N1)
/registros/<pk>/expediente/	GET	Línea de tiempo del expediente
/registros/<pk>/verificar/	GET	Verificar firma ECDSA del registro
/registros/workflow/	GET	Listado de solicitudes de cambio
/registros/workflow/<pk>/decide/	POST	Aprobar o rechazar solicitud
/registros/workflow/<pk>/execute/	POST	Ejecutar solicitud aprobada
/registros/arco/	GET	Listado de solicitudes ARCO
/registros/<pk>/arco/nueva/	GET, POST	Nueva solicitud ARCO para un registro
/registros/arco/<pk>/	GET	Detalle de solicitud ARCO
/registros/arco/<pk>/ejecutar/	POST	Ejecutar solicitud ARCO
/registros/cadena/	GET	Visor de la cadena de auditoría criptográfica
/registros/tickets/	GET	Listado de tickets del sistema

8.3 Ejemplo de uso: crear registro vía vistas

```
# Fragmento simplificado de views.py en registros/
from crypto_core.signatures import ECDSASigner
from crypto_core.hashing import sha256_hex

def registro_review(request):
    # 1. Recuperar datos del formulario (guardados en sesion)
    data = request.session.get('registro_draft')
    # 2. Firmar con la clave del usuario autenticado
    payload = json.dumps(data, sort_keys=True)
    msg_hash = sha256_hex(payload.encode())
    signer = ECDSASigner(private_key_pem=request.session['private_key_pem'])
    r, s = signer.sign(payload.encode())
    # 3. Crear el registro y su firma
    reg = MigrantRegistration.objects.create(**data, created_by=request.user)
    MigrantRegistrationSignature.objects.create(
        registration=reg, message_hash=msg_hash,
        signature_r=str(r), signature_s=str(s),
        public_key=signer.public_key_pem(), curve_name='secp256k1',
        signed_by=request.user, signed_by_role=request.user.access_level,
    )
```

Flujos y Lógica de Negocio

9.1 Flujo de autenticación (login)

1. El usuario envía `username + password + (para N1/N2) los archivos .cert y .key`.
2. El sistema verifica el límite de intentos fallidos: si hay 5 o más en los últimos 15 minutos (`LoginAttempt.is_blocked`), rechaza con código 429.
3. Autenticación Django con `authenticate()`. Si falla, registra un `LoginAttempt(successful=False)`.
4. Si el usuario tiene `is_revoked=True` o `is_deleted=True`: error “Cuenta inactiva o revocada”.
5. Si el `onboarding_status` es `pending`: redirige a `/iam/onboarding/`.
6. Para N1/N2: verifica que el `.cert` presentado coincide con el `fingerprint` almacenado en `UserCertificate` y que el certificado no está revocado ni expirado.
7. Si el usuario tiene MFA activo: solicita el código TOTP; se verifica con `pyotp.TOTP.verify`.
8. Sesión iniciada; registra `LoginAttempt(successful=True)` y entrada en `AuditLog`.

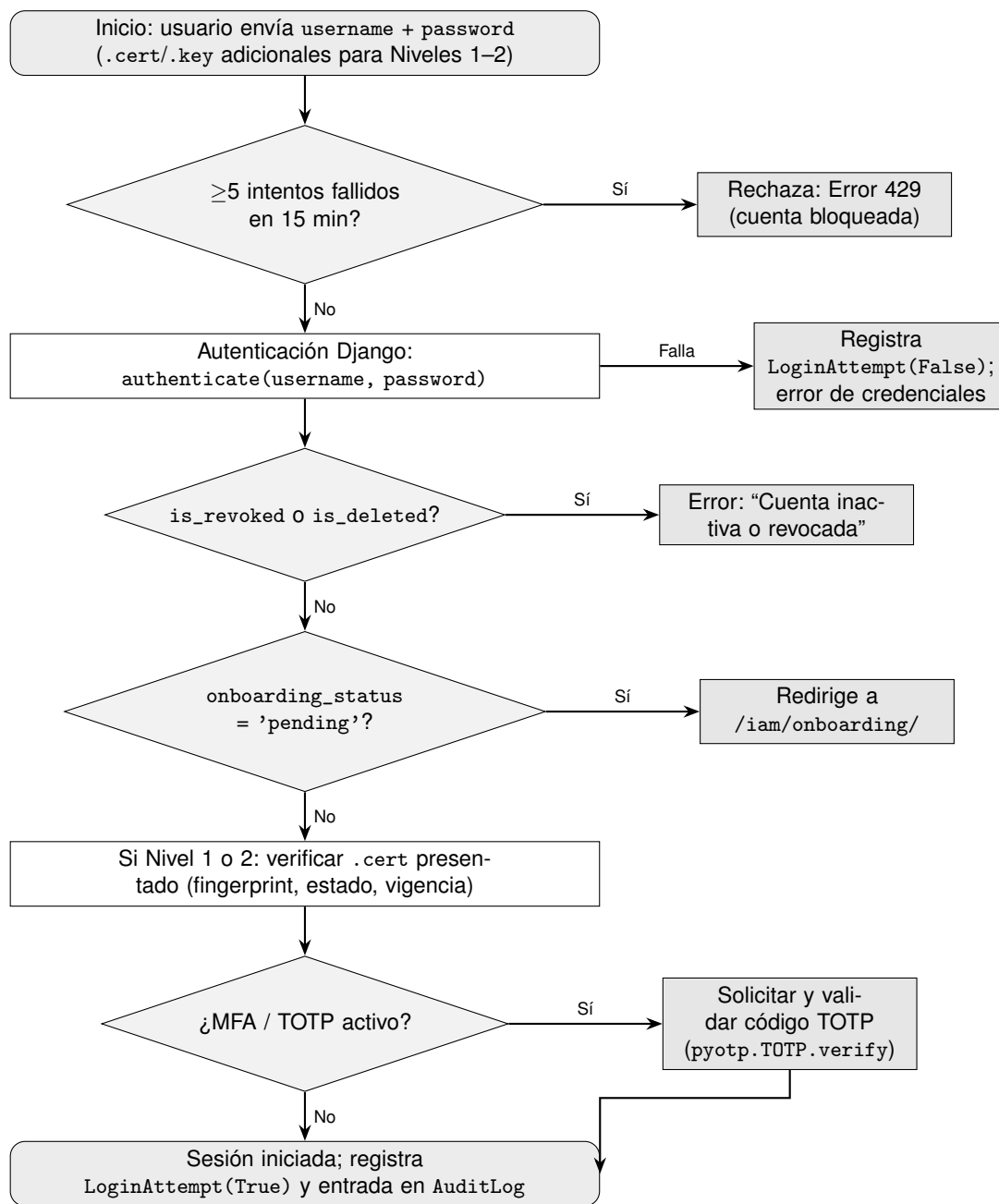


Figura 2: Flujo de autenticación del sistema. Las validaciones se ejecutan en secuencia: (1) bloqueo por exceso de intentos fallidos (Error 429), (2) autenticación de credenciales con `authenticate()` de Django, (3) verificación del estado activo de la cuenta, (4) redirección al onboarding si está pendiente, (5) comprobación del certificado X.509 únicamente para Niveles 1–2 (fingerprint, estado y vigencia), y (6) validación del código TOTP cuando MFA está habilitado. Cada rama de error termina el proceso; la rama “No” en cada decisión continúa el flujo principal. Elaboración propia.

9.2 Flujo de onboarding de nuevo colaborador

1. Admin crea el colaborador con contraseña temporal (`onboarding_status='pending'`).
2. El colaborador ingresa y completa el formulario de onboarding: datos personales, ID oficial, pregunta de seguridad.
3. El estado cambia a `submitted`; el Admin visualiza la solicitud en `/iam/onboarding/pending/`.

4. Admin aprueba: estado $\rightarrow$ approved; si nivel ≤ 2 , el sistema emite automáticamente el certificado X.509 y el par de claves.
5. El colaborador descarga sus archivos .cert y .key; los campos cert_file_downloaded y key_file_downloaded se marcan en True.
6. En el siguiente login el colaborador presenta sus credenciales digitales junto con usuario y contraseña.

9.3 Flujo del workflow de aprobación

Cuadro 10: Reglas de escalación por tipo de acción (elaboración propia).

Tipo de acción	Solicitante	Cadena de aprobación
Crear registro	N4	N3 revisa $\rightarrow$ N2 aprueba $\rightarrow$ N1 ejecuta
Actualizar registro	N3/N4	N2 aprueba $\rightarrow$ N1 ejecuta
Eliminar registro	Cualquiera	Escala hasta N1
ARCO Acceso/Oposición	N3+	N2 autoriza $\rightarrow$ N1 ejecuta
ARCO Rectificación	N3+	N2 ejecuta
ARCO Cancelación	Cualquiera	N1 ejecuta únicamente

Los estados del workflow son: submitted $\rightarrow$ pending_review $\rightarrow$ escalated $\rightarrow$ approved $\rightarrow$ signed $\rightarrow$ executed (o rejected en cualquier punto). Cada transición de estado requiere la firma digital del actor.

9.4 Reglas importantes

- **Inmutabilidad:** los modelos CertificateAuditLog, SignedFlowLog y los ApprovalStep no permiten UPDATE ni DELETE a nivel de modelo Python (el método save() lanza PermissionError si ya tiene PK).
- **Cifrado transparente:** los campos declarados como EncryptedTextField o EncryptedDateField se cifran automáticamente al escribir en la BD y se descifran al leer. El código de las vistas trabaja siempre con valores en claro.
- **Eliminación lógica:** ningún expediente se borra físicamente de la BD. El campo is_deleted=True oculta el registro del listado público; solo el administrador puede verlo en /registros/eliminados/.
- **Plazo ARCO:** al crear un ArcoRequest, el sistema calcula automáticamente legal_deadline = fecha_recepción + 20 días hábiles.
- **Identificadores únicos:** Collaborator.internal_id sigue el patrón CM-XXXXXXX; MigrantRegistration.internal_id sigue MIG-XXXXXXX; ArcoRequest.case_id sigue ARCO-XXXXXXX. Se generan en save() si están vacíos.

Ejemplos de Uso

10.1 Crear un colaborador desde código

```
from iam.models import Collaborator, Area

area = Area.objects.get(slug='humanitaria')
colaborador = Collaborator.objects.create_user(
    username='jdoe',
    email='jdoe@casamonarca.org',
    password='TempPass2026!',
    access_level=3,
    area=area,
    role='Operativo Humanitario',
)
# El internal_id se asigna automaticamente en save()
print(colaborador.internal_id) # CM-A1B2C3D4
```

10.2 Emitir un certificado X.509

```
from crypto_core.ca import CertificateAuthority
from iam.models import UserCertificate

ca = CertificateAuthority()
cert_pem, key_pem, fingerprint = ca.issue_certificate(
    subject_name=colaborador.get_full_name(),
    days_valid=365,
)
UserCertificate.objects.create(
    collaborator=colaborador,
    certificate_data=cert_pem,
    key_data=key_pem, # cifrado con CERT_ENCRYPTION_KEY
    fingerprint=fingerprint,
    expires_at=timezone.now() + timedelta(days=365),
    issued_by=admin_user,
)
```

10.3 Firmar y verificar un expediente

```
from crypto_core.signatures import ECDSASigner
import json

# Firma
payload = json.dumps({'internal_id': 'MIG-00000001',
                      'first_name': 'Juan'}, sort_keys=True)
signer = ECDSASigner(private_key_pem=key_pem)
r, s = signer.sign(payload.encode())

# Verificacion
verifier = ECDSASigner(public_key_pem=signer.public_key_pem())
valid = verifier.verify(payload.encode(), r, s)
print(valid) # True
```

10.4 Consultar el estado de la cadena de auditoría

```
from registros.models import ActionSignature

# Verificar que la cadena no ha sido manipulada
entries = ActionSignature.objects.order_by('chain_position')
prev_hash = ''
for entry in entries:
    if entry.prev_chain_hash != prev_hash:
        print(f"CADENA ROTA en posicion {entry.chain_position}")
        break
    prev_hash = entry.chain_hash()
else:
    print("Cadena integra.")
```

Manejo de Errores

11.1 Estrategia de errores

El sistema sigue la estrategia de Django para el manejo de errores HTTP:

- **403 Forbidden:** cuando un usuario intenta acceder a una vista para la que no tiene el nivel de acceso requerido. Django devuelve la plantilla `403.html`.
- **404 Not Found:** cuando se solicita un objeto con un ID que no existe (`get_object_or_404`).
- **500 Internal Server Error:** errores no capturados. En producción (`DEBUG=False`), Django registra el error y muestra una página genérica.

Para errores de negocio (formularios inválidos, firma incorrecta, certificado expirado), las vistas retornan el formulario con mensajes de error explícitos a través del sistema de mensajes de Django (`messages.error`).

11.2 Mensajes comunes

Cuadro 11: Mensajes de error frecuentes y su causa (elaboración propia).

Mensaje	Causa	Acción
<i>"Usuario o contraseña incorrectos"</i>	Credenciales inválidas	Verificar datos; revisar <code>LoginAttempt</code>
<i>"Cuenta inactiva o revocada"</i>	<code>is_revoked=True</code> o <code>is_deleted=True</code>	Contactar al administrador
<i>"Certificado inválido o expirado"</i>	<code>UserCertificate</code> revocado o <code>expires_at</code> pasado	Solicitar renovación
<i>"Debes aceptar el Aviso de Privacidad"</i>	<code>data_consent=False</code> en el formulario	Marcar el campo de consentimiento
<i>"Los menores no pueden superar el tamaño del grupo"</i>	Validación de consistencia del registro	Revisar los campos de grupo y menores
<code>CERT_ENCRYPTION_KEY</code> must be set in production	Variable de entorno faltante	Configurar <code>CERT_ENCRYPTION_KEY</code> en producción

11.3 Registro (logs)

- `AuditLog`: registra todas las acciones IAM con `actor`, `target`, `action` y `created_at`. Visible en `/iam/audit/`.
- `LoginAttempt`: registra cada intento de inicio de sesión con IP, usuario y resultado. Permite detectar fuerza bruta.
- `RegistrationEvent`: bitácora por expediente (consultas, ediciones, eventos ARCO). Visible en `/registros/<pk>/expedi`.
- `CertificateAuditLog`: historial de cambios de estado de cada certificado. Append-only.
- En producción, Django escribe errores no capturados en el log del servidor (configurado en `LOGGING` dentro de `settings.py`).

Pruebas

12.1 Tipos de tests

El proyecto incluye pruebas de integración criptográfica ubicadas en `tests/test_crypto_integration.py`. Estas pruebas validan:

- Cifrado y descifrado simétrico con `EncryptionService` (AES-256-GCM).
- Generación, firma y verificación ECDSA con `ECDSASigner`.
- Emisión y verificación de certificados X.509 con `CertificateAuthority`.
- Integridad de la cadena de auditoría (`ActionSignature.chain_hash`).
- Funcionamiento de los campos personalizados `EncryptedTextField` y `EncryptedDateField` sobre una base de datos SQLite en memoria.

12.2 Cómo ejecutar las pruebas

```
# Instalar dependencias de prueba
pip install pytest pytest-django

# Ejecutar toda la suite
pytest

# Pruebas con salida detallada
pytest tests/ -v

# Solo pruebas de un archivo
pytest tests/test_crypto_integration.py -v

# Con informe de cobertura (requiere pytest-cov)
pip install pytest-cov
pytest --cov=crypto_core --cov-report=term-missing
```

La configuración de `pytest` se lee desde `pytest.ini` o `pyproject.toml` en la raíz; la variable `DJANGO_SETTINGS_MODULE` debe apuntar a `casamonarca.settings`.

Despliegue

13.1 Entornos

Cuadro 12: Entornos de despliegue (elaboración propia).

Entorno	Base de datos	Servidor
Desarrollo local	SQLite (automático)	<code>python manage.py runserver</code>
Staging / QA	MySQL 8	Gunicorn + nginx en VPS
Producción	MySQL 8 en HostGator	Gunicorn configurado en cPanel

13.2 Cómo desplegar en producción (HostGator)

HostGator es un servicio de hosting compartido. No soporta Docker, Java ni contenedores. El despliegue se realiza mediante cPanel con Python Application Manager.

Paso 1 — Configurar Python Application en cPanel:

```
Aplicacion: Python 3.10
Directorio de la aplicacion: /home/<usuario>/casamonarca
Startup file: passenger_wsgi.py
```

Paso 2 — Subir el código:

```
git clone <repo> /home/<usuario>/casamonarca/CriptoReto
```

Paso 3 — Instalar dependencias en el entorno virtual de cPanel:

```
pip install -r requirements.txt
```

Paso 4 — Configurar variables de entorno en cPanel → Entorno de la aplicación.

Paso 5 — Aplicar migraciones y coleccionar estáticos:

```
python manage.py migrate
python manage.py collectstatic --noinput
```

Paso 6 — Reiniciar la aplicación desde cPanel.

13.3 Consideraciones de producción

- Establecer `DEBUG=False` y configurar `ALLOWED_HOSTS` con el dominio real.
- Generar una nueva `SECRET_KEY` única para producción.
- Configurar `CERT_ENCRYPTION_KEY` con una clave aleatoria de 32 bytes (`secrets.token_urlsafe(32)`).
- Subir `ecc_signing_key.pem` y el certificado de la CA a rutas fuera del `DOCUMENT_ROOT` y configurar `ECC_PRIVATE_KEY_PATH` y `CERT_PRIVATE_KEY_PATH`.
- Activar HTTPS; la comunicación entre el cliente y el servidor debe estar cifrada en tránsito.
- Configurar backups automáticos de la base de datos MySQL desde cPanel.

Seguridad

14.1 Autenticación

- **Contraseña:** almacenada con el hasher PBKDF2-SHA256 de Django (`make_password`).
- **Certificado digital X.509 (Niveles 1 y 2):** en cada login se verifica que el certificado presentado coincide con el almacenado, no está revocado y no ha expirado.
- **TOTP (MFA):** código de un uso de 6 dígitos validado con `pyotp.TOTP.verify` con ventana de ± 1 período (30 s).
- **Brute-force protection:** 5 intentos fallidos en 15 minutos bloquean temporalmente la cuenta (`LoginAttempt.is_blocked`).
- **Pregunta de seguridad:** respuesta almacenada con `make_password`; usada solo para recuperación de contraseña.

14.2 Manejo de datos sensibles

- Los campos PII de `MigrantRegistration` (nombre, apellidos, fecha de nacimiento, teléfono, país de origen) se cifran con AES-256-GCM antes de almacenarse. La clave de cifrado proviene de `CERT_ENCRYPTION_KEY`.
- Los datos del certificado X.509 y la clave privada cifrada en `UserCertificate` también se almacenan cifrados.
- Los archivos subidos por usuarios (IDs oficiales, documentos ARCO) se guardan en `media/` con nombres aleatorizados; no deben servirse directamente desde nginx sin autenticación.
- El archivo `.env` y las claves PEM nunca deben subirse al repositorio de código.

14.3 Buenas prácticas implementadas

- **CSRF:** todas las peticiones POST están protegidas por el middleware CSRF de Django.
- **XSS:** las plantillas Django escapan automáticamente el contenido dinámico (`{{ variable }}`).
- **SQL Injection:** todo acceso a la base de datos se realiza a través del ORM de Django; no se ejecuta SQL crudo.
- **Clickjacking:** el middleware `XFrameOptionsMiddleware` de Django incluye la cabecera `X-Frame-Options: DENY`.
- **Inmutabilidad de registros críticos:** `SignedFlowLog`, `CertificateAuditLog` y `ApprovalStep` no pueden modificarse ni eliminarse a nivel ORM.
- **Principio de mínimo privilegio:** cada vista comprueba el `access_level` del usuario antes de ejecutar cualquier acción. Los niveles 3 y 4 no pueden ejecutar acciones directas sobre expedientes; solo solicitar.

Guía de Contribución

15.1 Convenciones de código

- Seguir **PEP 8** para Python; máximo 100 caracteres por línea.
- Los nombres de modelos en singular (*Collaborator*, no *Collaborators*).
- Los nombres de vistas en formato `<entidad>_<accion>` (`registro_list`, `registro_edit`).
- Documentar funciones complejas con docstring de una línea que explique el “por qué”.
- No escribir SQL crudo; usar siempre el ORM.
- No almacenar claves criptográficas en el código ni en `settings.py`; usar variables de entorno.

15.2 Flujo de trabajo con git

```
# Crear rama desde main para cada funcionalidad
git checkout -b feature/<nombre-corto>

# Commits atomicos con mensaje descriptivo
git commit -m "feat(iam): agregar campo TOTP a Collaborator"

# Pull request hacia main; revisar con al menos un compañero
# No hacer push directo a main
```

Convención de prefijos en commits: `feat` (nueva funcionalidad), `fix` (corrección), `refactor`, `test`, `docs`.

15.3 Cómo agregar una nueva funcionalidad

1. Si requiere nuevos datos: agregar el campo al modelo correspondiente, generar la migración (`makemigrations`) y aplicarla (`migrate`).
2. Crear o actualizar la vista en `views.py`; verificar el nivel de acceso requerido al inicio de la función.
3. Registrar la URL en `urls.py` con un nombre descriptivo.
4. Crear o actualizar la plantilla HTML en `templates/<app>/`.
5. Si la funcionalidad involucra criptografía: implementarla en `crypto_core/` y agregar pruebas en `tests/test_crypto_inte`.
6. Registrar acciones relevantes en `AuditLog` o `RegistrationEvent` según corresponda.

Problemas Conocidos

16.1 Bugs actuales

Al tratarse de la primera versión del sistema, no se han identificado bugs críticos. Los casos de prueba de los módulos IAM y Registro de Migrantes resultaron exitosos (ver tabla completa en el Reporte Técnico, Anexo A).

16.2 Limitaciones

- **HostGator (hosting compartido):** no admite Docker, contenedores, ni servicios adicionales como Java. Esto descarta el uso de Keycloak o step-ca en el servidor actual de Casa Monarca.
- **Sin notificaciones por correo:** el sistema genera notificaciones internas, pero no envía emails automáticos al ejecutar acciones relevantes. Esta funcionalidad está identificada como mejora futura.
- **Sin respaldo automático:** no existe un cron job de backup de la base de datos. Se recomienda configurar uno desde cPanel o contratar el servicio de backups de HostGator.
- **CA autofirmada:** el sistema actúa como su propia autoridad certificadora. Los certificados emitidos no son reconocidos por navegadores externos ni por terceros fuera del sistema.
- **Un único servidor ECC:** la clave privada del servidor (`ecc_signing_key.pem`) se usa para todas las firmas del sistema. Si esta clave se compromete, se debe regenerar y refirmar todos los registros.

FAQ Técnica

¿Por qué el servidor no inicia y lanza ImproperlyConfigured: CERT_ENCRYPTION_KEY must be set in production? Porque DEBUG=False y la variable CERT_ENCRYPTION_KEY no está definida. Generarla con:

```
python -c "import secrets, base64; print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode())"
```

y agregarla al entorno de producción.

¿Cómo regenerar la base de datos desde cero en desarrollo?

```
rm db.sqlite3
python manage.py migrate
python manage.py createsuperuser
python seed.py # opcional: datos de prueba
```

Las migraciones de iam fallan con Table 'iam_collaborator' already exists. Si la tabla ya existe en MySQL (de una migración parcial), marcar la migración como aplicada sin ejecutarla:

```
python manage.py migrate iam --fake-initial
```

¿Cómo verificar que la cadena de auditoría no ha sido manipulada? Ejecutar el verificador desde el shell de Django:

```
python manage.py shell
>>> from registros.models import ActionSignature
>>> entries = ActionSignature.objects.order_by('chain_position')
>>> prev = ''
>>> ok = all((e.prev_chain_hash == prev) and (prev := e.chain_hash()) or True for e in entries)
>>> print('Cadena integra' if ok else 'CADENA COMPROMETIDA')
```

Un usuario de Nivel 2 no puede ver los registros de migrantes. Verificar que el usuario tiene onboarding_status='approved' y que su area está asignada. El filtro de visibilidad en las vistas usa user.area para restringir los registros que puede ver un Nivel 2.

La firma ECDSA falla con InvalidSignature. Las causas más comunes son: (1) la clave privada no corresponde al certificado presentado; (2) el payload fue serializado con distinto orden de llaves. Verificar que el campo curve_name es secp256k1 y que el json.dumps usa sort_keys=True.

Referencias

- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>
- Python Cryptographic Authority. (2024). *cryptography 42.0 documentation*. <https://cryptography.io/en/latest/>
- HostGator. (2026). *Python Application Setup in cPanel*. <https://support.hostgator.com/>
- PyPI. (2026). *pyotp — Python One Time Password Library*. <https://pypi.org/project/pyotp/>
- PyPI. (2026). *python-dotenv*. <https://pypi.org/project/python-dotenv/>
- National Institute of Standards and Technology. (2023). *FIPS 186-5: Digital Signature Standard (DSS)*. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>
- Boeyen, S., Santesson, S., Polk, T., Housley, R., Farrell, S., & Cooper, D. (2008). *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile* (RFC 5280). <https://datatracker.ietf.org/doc/rfc5280/>
- Cámara de Diputados del H. Congreso de la Unión. (2010). *Ley Federal de Protección de Datos Personales en Posesión de Particulares*. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>